



## HTML5 + CSS3 Implementation

Building a responsive student productivity workspace with modern web fundamentals.

 HTML5

 CSS3

# From markup to interface



## HTML5

Structure — what the content is



## CSS3

Presentation — how it looks



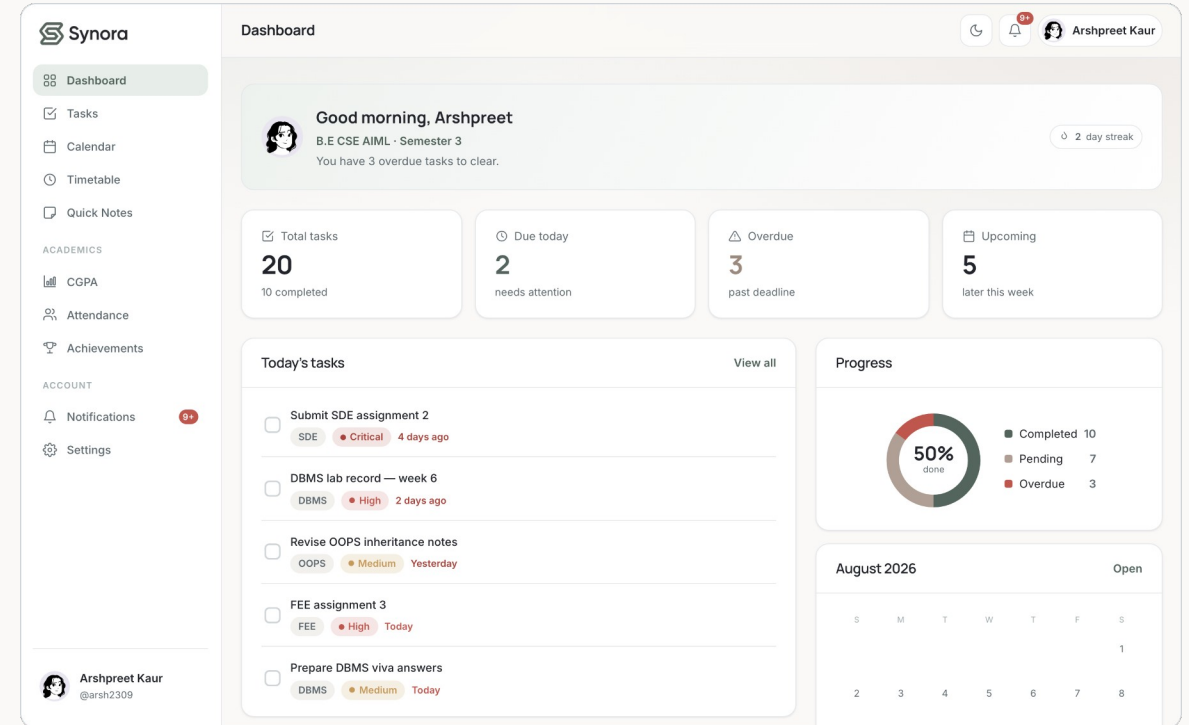
## Responsive

Adapts to every screen



## Synora UI

The finished interface



*HTML gives the page meaning; CSS gives it the calm, consistent look of the Synora dashboard.*

**Together** HTML5 + CSS3 form the entire foundation of Synora's interface.

# The document foundation

```
index.html

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport"
6     content="width=device-width, initial-scale=1">
7   <title>Synora – One calm home...</title>
8   <meta name="color-scheme" content="light dark">
9   <link rel="stylesheet" href="styles/tokens.css">
10  <link rel="stylesheet" href="styles/app.css">
11 </head>
12 <body class="app" data-page="dashboard">
13   ...
14 </body>
15 </html>
```

**<!DOCTYPE html>**

declares HTML5 — standards mode

**lang="en"**

document language for screen readers

**meta charset**

UTF-8 text encoding

**meta viewport**

the key to responsive design

**link stylesheet**

external CSS, loaded in order

**Why?** The <meta viewport> tag tells mobile browsers to use the device width — without it, media queries and mobile layouts don't work.

# Semantic structure

```
index.html – landing structure

1 <header class="site-header">
2   <nav aria-label="Primary"> ... </nav>
3 </header>
4 <main id="main">
5   <section class="hero"> ... </section>
6   <section id="features"> ... </section>
7 </main>
8 <footer class="site-footer">
9   <nav aria-label="Footer"> ... </nav>
10 </footer>
```

**Also used:** `<article>` wraps each attendance subject card — a self-contained unit of content.

## THE TAGS SYNORA USES

`<header>``<nav>``<main>``<section>``<article>``<footer>`

**Why?** Semantic tags give the document a meaningful hierarchy — better accessibility, easier maintenance, and clear DOM targeting — instead of a wall of `<div>`s.

# Accessibility & forms

```
index.html – Get in Touch

1 <form class="contact-form" novalidate>
2   <div class="field">
3     <label for="contact-email">Email</label>
4     <input type="email" id="contact-email"
5           name="email" autocomplete="email"
6           placeholder="you@example.com" required>
7   </div>
8   <button type="submit">Send Message</button>
9 </form>
```

## ACCESSIBILITY PRACTICES IN SYNORA

- <label for> tied to every input id
- Real <button>s, never clickable <div>s
- Typed inputs — email, tel — aid entry & keyboards
- Decorative SVGs marked aria-hidden
- aria-label on icon-only controls; aria-live toasts
- Visible keyboard focus rings

**HTML Form** → User input → Validation attributes (required, type) → handed to JavaScript

**Honest scope** Synora follows solid accessibility fundamentals (semantics, labels, focus, contrast) — it has not been formally WCAG-audited, so it is described as accessible-by-construction, not certified.

# Building a design system

```
tokens.css + components.css

1 :root {
2   --color-brand: #53655C; /* sage */
3   --color-accent: #B09F95; /* taupe */
4   --color-bg: #FAF9F7;
5   --color-text: #22252B;
6   --space-24: 24px;
7   --radius-lg: 16px;
8 }
9 .btn--primary {
10  background: var(--color-brand);
11  padding: 0 var(--space-24);
12  border-radius: var(--radius-lg);
13 }
```

## ONE LANGUAGE, EVERY SCREEN



**Why?** CSS custom properties centralise every colour, space and radius as one reusable token set — so buttons, cards, forms and the dashboard all share one visual language, and dark mode is a single re-declaration.

# The Box Model



```
base.css + app.css

1 *, *::before, *::after {
2   box-sizing: border-box; /* base.css */
3 }
4 .att-card {
5   padding: var(--space-16);
6   border: 1px solid var(--color-border);
7   border-radius: var(--radius-lg);
8   gap: var(--space-12);
9 }
```

**Why?** `box-sizing: border-box` makes width include padding and border — so a card is exactly the size you set, and layouts never break from added padding. Margin is space outside the element; padding is space inside, around the content.

# Flexbox — one-dimensional layout

```
app.css

1 .topbar {
2   display: flex;
3   align-items: center;
4   gap: var(--space-16);
5 }
6 .card__header {
7   display: flex;
8   align-items: center;
9   justify-content: space-between;
10  flex-wrap: wrap;
11 }
```

MAIN AXIS →

Logo

Title

Actions

*justify-content spreads • align-items centres • gap spaces*

**Used for** the top bar, card headers, button groups, steppers — anything laid out along a single row or column.

**Why?** Flexbox aligns and distributes items along one axis with almost no math — ideal for toolbars and headers where content sizes vary.

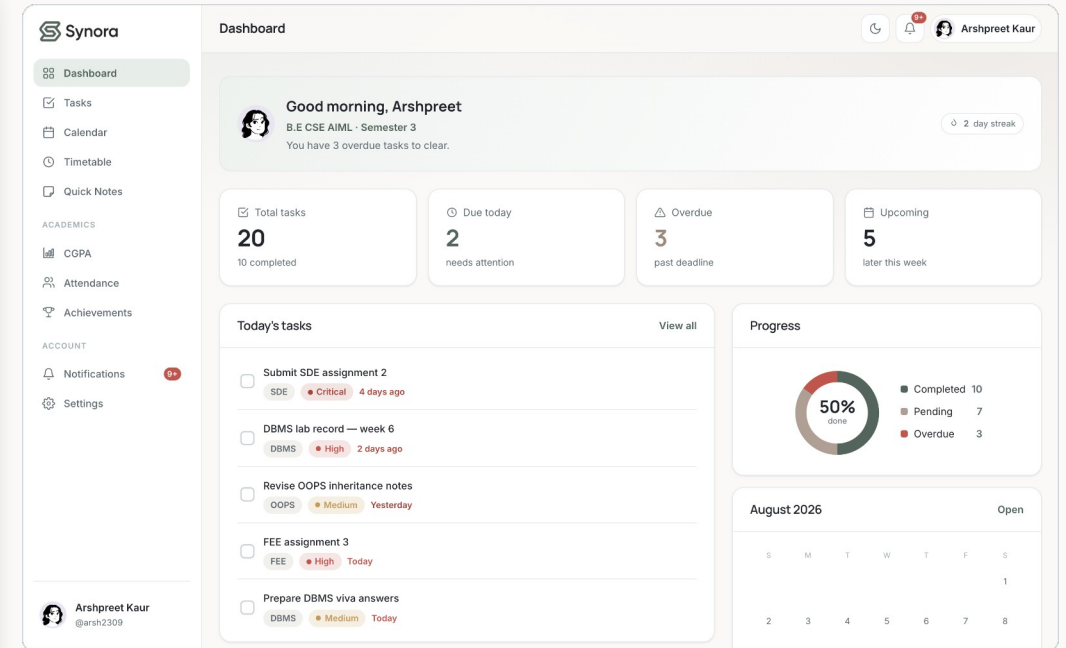
# CSS Grid — two-dimensional layout

```

app.css — dashboard

1 .dash-cols {
2   display: grid;
3   grid-template-columns: 1fr; /* stacked */
4 }
5 @media (min-width: 1000px) {
6   .dash-cols {
7     grid-template-columns:
8       minmax(0,1.6fr) minmax(0,1fr);
9   }
10 }
11 .stat-grid {
12   grid-template-columns:
13     repeat(auto-fit, minmax(150px, 1fr));
14 }

```



The dashboard: a grid of cards that reflow by column.

**Why?** Grid controls rows and columns at once — perfect for the dashboard. `repeat(auto-fit, minmax())` even reflows the stat cards responsively with no media query.

# Choosing the right layout



## Flexbox

One-dimensional — a row OR a column



## CSS Grid

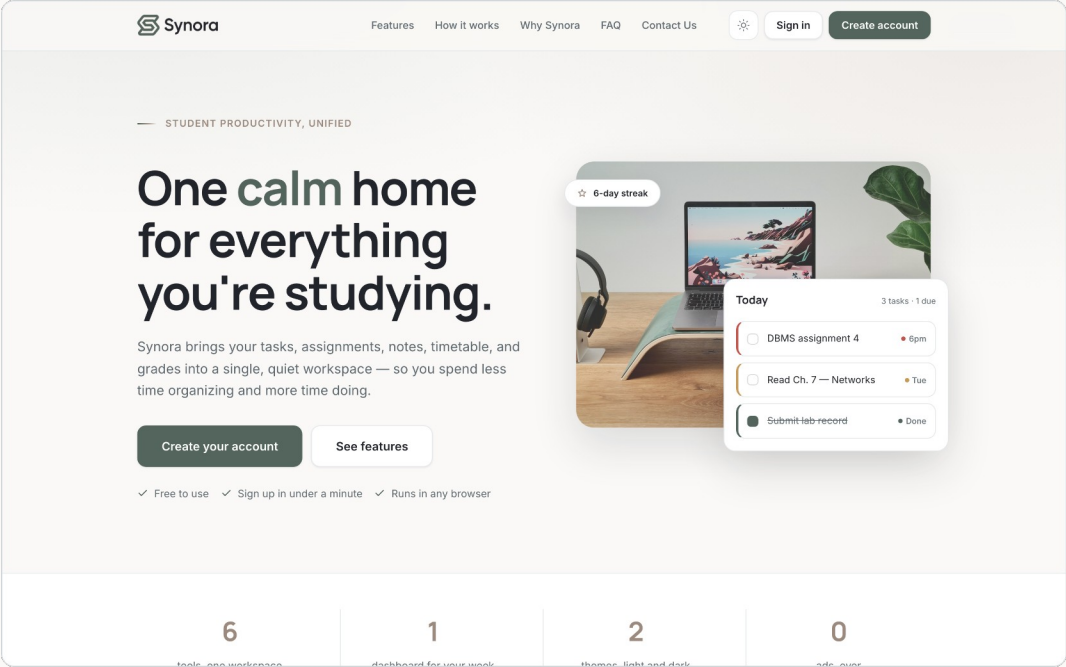
Two-dimensional — rows AND columns

|              |                                                   |                                                   |
|--------------|---------------------------------------------------|---------------------------------------------------|
| Direction    | Align along one axis                              | Structure across two axes                         |
| Best for     | Navbars, headers, controls                        | Dashboards, card layouts, pages                   |
| In Synora    | <code>.topbar</code> · <code>.card__header</code> | <code>.dash-cols</code> · <code>.stat-grid</code> |
| Content flow | Sizes to its content                              | Defines the track structure first                 |

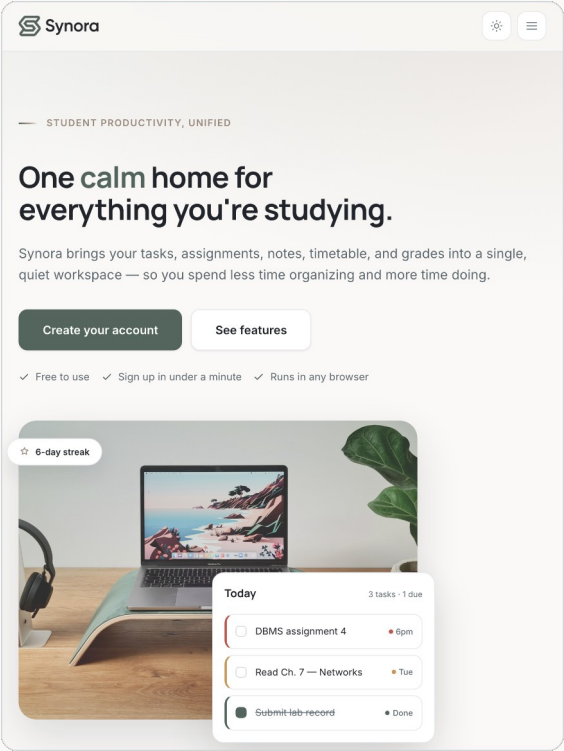
**Rule of thumb:** *content in a line* → Flexbox; *a structured grid of things* → Grid.

# One workspace, every screen

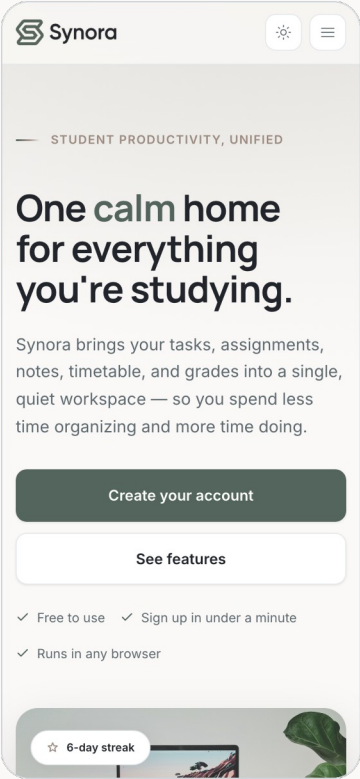
The same landing page at three real viewports. The full navbar collapses into a CSS-only hamburger menu, the two-column hero stacks into one, and every section reflows to a single column.



1440px • Desktop



768px • Tablet



390px • Mobile

Fluid units

Flexbox

CSS Grid

Media queries

# Media queries — adapting the layout

```
app.css

1 /* below 900px: sidebar becomes a drawer */
2 @media (max-width: 900px) {
3   .sidebar {
4     position: fixed;
5     transform: translateX(-100%);
6   }
7   .app.nav-open .sidebar {
8     transform: translateX(0);
9   }
10  .menu-btn { display: inline-flex; }
11 }
12 /* below 560px: 7-col timetable stacks */
13 @media (max-width: 560px) {
14   .tt-grid { grid-template-columns: 1fr; }
15 }
```

## REAL BREAKPOINTS IN SYNORA

### 1024 / 940 / 720px

Landing sections reflow to fewer columns

### 900px

App sidebar → off-canvas drawer + hamburger

### 560px

Timetable grid → stacked day cards

### 1000px (min)

Dashboard gains its two-column split

# The responsive strategy — honestly



## Predominantly desktop-first

Base (unqualified) styles describe the full desktop layout; max-width media queries progressively adapt it down.

**Desktop base** → `@media (max-width: 900px)` → `@media (max-width: 560px)`

*44 of Synora's media queries use max-width — the graceful-degradation pattern.*



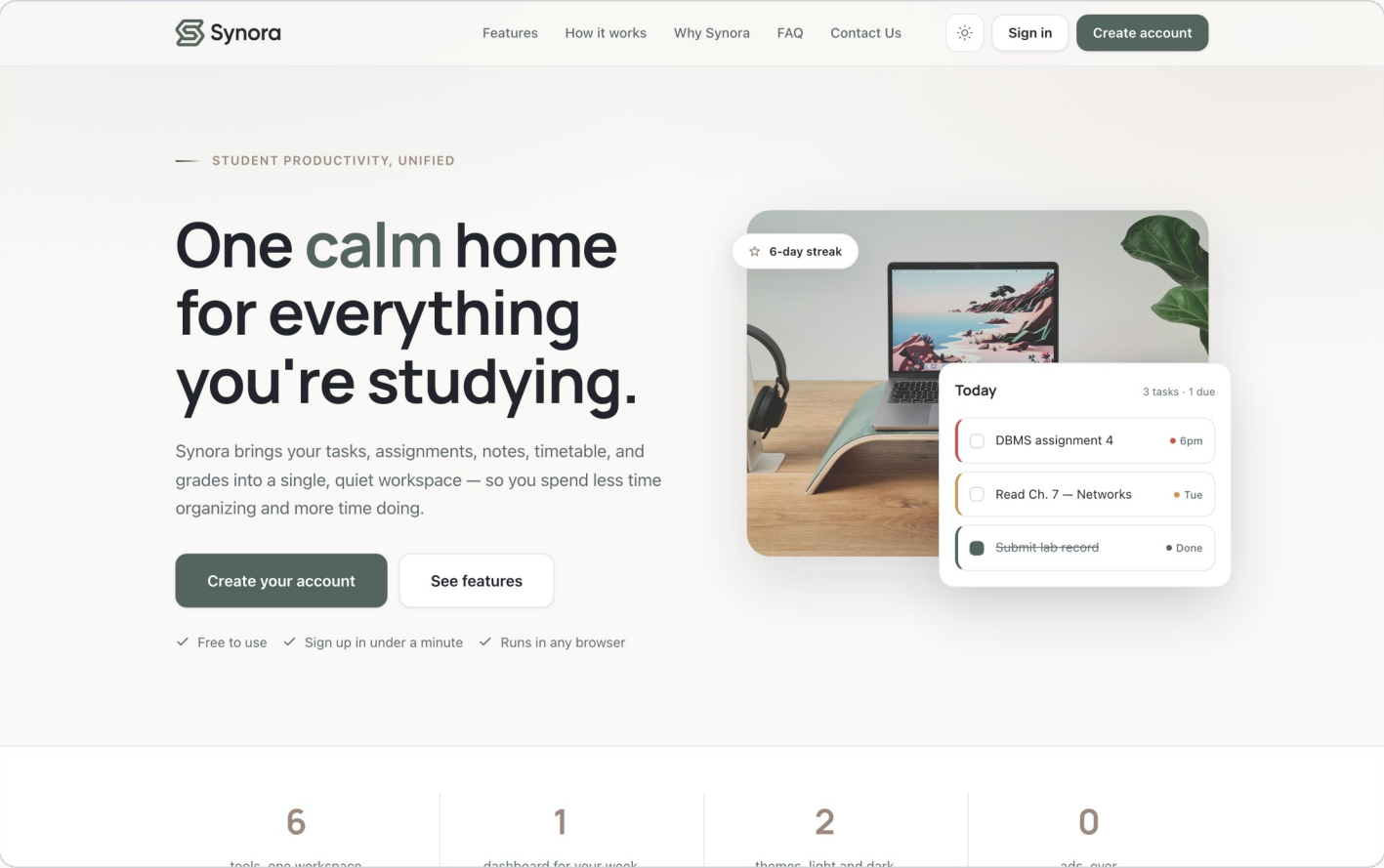
## With mobile-first components

Several parts are authored the other way — stacked by default, expanding upward — or reflow with no query at all.

- `.dash-cols` — 1 column, then min-width: 1000px
- `.stat-grid` — `repeat(auto-fit, minmax())` reflows freely
- Fluid type with `clamp()` in the tokens

**Why?** Stating this correctly is a viva safeguard: Synora is not a pure mobile-first cascade — it is breakpoint-based adaptation from a desktop base, with intrinsically-responsive components layered in. Naming it accurately shows you understand the difference.

# The Synora landing page



**HTML5**  
Semantic header / nav / main / section / footer



**CSS3**  
Design tokens, typography & the box model



**Flexbox**  
Nav actions, hero and footer rows



**CSS Grid**  
Centered navbar & the feature sections



**Media queries**  
Reflows at 1024 / 940 / 720 / 400px

The real landing page — the syllabus’ “static responsive landing page”, grown into a product’s marketing entry point.

# What we learned



## HTML5

Structure & meaning



## Semantic HTML

Accessible, maintainable markup



## CSS3

A token-driven design system



## Flexbox + Grid

One- & two-dimensional layout



## Media queries

Adapting to every screen



## Responsive design

One workspace, any device



*One calm home for everything you're studying.*